RegexLab Compiler Design Study Notes

Parser Pass and Recursive Descent

Compiler Design and Theory of Computation perspective

Source focus: engine/passes/parser-pass.cpp and parser-pass.hpp. The TypeScript parser in engine/passes/parser-pass.ts follows the same structure.

1. What the parser pass does

The parser converts regular-expression text into an Abstract Syntax Tree (AST). It does not build an automaton. Thompson's construction consumes the AST in the next pass and builds the ε-NFA.

The parser reads characters directly from left to right. It does not first run a separate lexer to produce a token stream. Its result is either a structured ParseResult containing an AST or a ParseError with a type and source position.

C++ entry point: parse(pattern, limits) in parser-pass.cpp. Its public declaration and defaults are in parser-pass.hpp.

2. Grammar supported by the current parser

```
regex ::= term ('|' term)*
term ::= factor+
factor ::= atom '*'?
atom ::= ASCII character | '(' regex ')'
```

In this grammar, the question mark after '*' means "zero or one star operator"; it does not mean the regex question-mark operator is implemented.

Concatenation is implicit: ab means a followed by b. Backslash consumes the next ASCII character as a literal. In the current implementation, |, *, parentheses, and backslash have syntax roles. Other ASCII characters are literals, so +, ?, ., [, and] are not operators.

3. Recursive descent in this code

Recursive descent means the parser functions follow the grammar from its highest-level rule down to its smallest units. The function call structure also establishes precedence.

```
parseRegex() handles alternation: a|b
parseTerm() handles concatenation: ab
parseFactor() handles optional *: a*
parseAtom() handles characters and groups
```

The parser is top-down: parseRegex asks for terms; a term asks for factors; a factor asks for an atom. Long chains of alternation and concatenation are processed with loops. Parentheses are where parseAtom recursively calls parseRegex to parse the group body.

4. Function-by-function

parse() — entry point

The public parse function constructs a Parser and calls its parse method. That method checks the pattern-length limit, calls parseRegex, then verifies that the whole pattern was consumed. An extra closing parenthesis at top level becomes UnmatchedParen; other leftover input becomes UnexpectedToken.

Expected syntax failures are raised internally as ParseFailure and caught at the top. They become ParseResult with ok=false and a ParseError. This keeps expected syntax errors structured instead of exposing exception text.

parseRegex() — alternation

It parses one term, then loops while the next character is |. Each iteration consumes the bar, parses another term, and builds an AltNode from the accumulated left side and the new right side. Thus a|b|c becomes alt(alt(a,b),c).

parseTerm() — implicit concatenation

It parses a factor, then repeatedly parses another factor while canStartFactor() says one can begin. The helper stops at end of input, |, or). Each factor is joined to the accumulated node with ConcatNode, so abc becomes concat(concat(a,b),c).

| ends a term because parseRegex handles it.) ends a term so the enclosing group parser can consume it. Other characters can start another factor; if that character is an invalid operator such as *, parseAtom reports the error.

parseFactor() — one optional star

It parses an atom first. If the next character is *, it consumes exactly one star and wraps the atom in a StarNode. For a**, the second star is encountered where an atom is required and produces DanglingOperator.

parseAtom() — character, group, or error

An ordinary ASCII character becomes a CharNode. On (, the parser remembers the opening position, consumes it, rejects an immediate) as EmptyGroup, increments the nesting counter, recursively parses the inner regex, requires a closing), consumes it, and returns the inner AST.

A |, *, or) where an atom is expected is DanglingOperator. Backslash consumes the next ASCII character as a literal; a trailing backslash is UnexpectedToken. End of input when an atom is required is also UnexpectedToken, as in an empty pattern or after a trailing |.

5. Precedence and parse examples

Precedence follows the call chain: parseRegex is loosest, parseTerm is tighter, parseFactor applies star, and parseAtom is tightest. Therefore * binds before concatenation, and concatenation binds before |.

Example: a|bc* —

Example: (a|b)*ab —

```
concat(
concat(
star(alt(char('a'), char('b'))),
char('a')
),
char('b')
)
```

This AST describes any sequence of a and b characters followed by ab: strings over {a,b} that end in ab.

6. Lookahead, errors, and limits

The Parser stores the source and a position index. peek() reads the next character without consuming it; advance() consumes it and increments the position. This small lookahead is enough to choose the grammar rule at each point.

a| ends while parsing the missing right-hand term and reports UnexpectedToken.

*a reports DanglingOperator because * has no atom to repeat.

() reports EmptyGroup.

(ab reports UnmatchedParen at the opening parenthesis.

ab) reports an extra closing parenthesis.

Default limits are 500 characters and group nesting depth 1000. The length cap also protects later recursive passes from deep ASTs built by long concatenation or alternation chains. Non-ASCII pattern characters are rejected. The cap applies to an all-ASCII prefix so UTF-8 bytes in C++ and UTF-16 code units in TypeScript are not incorrectly treated as the same length unit.

7. Compiler Design and TOC connection

In compiler-design terms, this is syntax analysis for a small source language: regular expressions. It recognizes operator structure and produces an AST intermediate representation. Error handling is lightweight syntax checking; there is no separate semantic-analysis pass, type checker, or symbol table.

In Theory of Computation terms, the AST records operations on regular languages: union, concatenation, and Kleene star. Thompson's construction can translate each AST node into an equivalent ε-NFA. The parser must preserve the user's intended grouping and precedence so the automaton recognizes the right language.

This is a hand-written recursive-descent parser, not a generated LL parser or an LR parser. The grammar is simple enough for direct character lookahead. The TypeScript implementation mirrors this structure; C++ is the production path compiled to WebAssembly.

8. Files to study alongside this explanation

engine/passes/parser-pass.cpp — Parser, ParseFailure, grammar functions, ASCII checks, and safety guards.

engine/include/regexlab/passes/parser-pass.hpp — parse interface, ParseLimits, and defaults.

engine/include/regexlab/types.hpp — ASTNode variants, ParseError, and ParseResult.

engine/passes/parser-pass.ts — TypeScript reference implementation.

tests/parser_pass_test.cpp — C++ tests for AST shapes, precedence, errors, and limits.

A useful study exercise is to trace one pattern through parseRegex → parseTerm → parseFactor → parseAtom while tracking the position index.

9. ThompsonPass — Thompson's construction

Compiler design and Theory of Computation perspective

Source focus: engine/passes/thompson-pass.cpp and engine/include/regexlab/passes/thompson-pass.hpp. The matching TypeScript reference is engine/passes/thompson-pass.ts. Tests are in tests/thompson_pass_test.cpp and engine/__tests__/unit/thompson-pass.test.ts.

9.1 What ThompsonPass does

ThompsonPass is the second construction stage after parsing. The parser turns the pattern into an AST; thompsonConstruction(ast) walks that tree and creates an ε-NFA (a nondeterministic finite automaton with epsilon transitions). It returns an Automaton, not a match result.

The NFA is an intermediate representation of the same regular language described by the AST. Later, SubsetConstructionPass removes nondeterminism by building DFA states from sets of NFA states and computing epsilon-closures. ThompsonPass itself does not run the input text through the automaton.

The implementation is a direct recursive translation: each AST node is converted into a small NFA fragment, and the fragments are joined according to the node's operator. Its four cases are character, concatenation, alternation, and Kleene star.

9.2 The AST and Automaton contracts

In engine/types.hpp, ASTNode contains a std::variant of CharNode, ConcatNode, AltNode, and StarNode. A character node stores one char; concat and alt each point to left and right child nodes; star points to one child. The parser guarantees the tree shape and precedence before this pass receives it.

Automaton is the common data shape used for the NFA, DFA, and minimized DFA. It has stateCount, startState, acceptStates, and transitions. A transition is indexed as source state → symbol → vector of destination states. A symbol is either an input character or the special key eps. The vector of destinations allows nondeterminism: on one symbol, a state may have more than one possible next state.

This pass returns one accept state, represented in the general-purpose acceptStates set as a singleton. The general Automaton type permits multiple accept states because later automata and passes share the same contract.

9.3 Fragment and builder: the central implementation idea

The private Fragment struct contains only two integers: start and accept. They identify the entry and exit boundary of the NFA built for one AST subtree. The fragment does not own a separate transition map; all fragments are assembled into one NFABuilder.

The key invariant is that a completed subtree has a designated start and accept boundary. The parent can connect to the child at those boundaries without needing to know the child's internal states or transitions. When a child fragment is embedded in a parent, the parent may add edges at those boundary states; the completed outer fragment again exposes its own start and accept.

NFABuilder owns the mutable construction state. newState() returns the current stateCount and increments it, producing IDs 0, 1, 2, and so on. addTransition(from, symbol, to) appends the destination to transitions_[from][symbol]. build(start, accept) packages the accumulated count and transitions into an Automaton and puts accept in its accept-state set.

buildFragment(node, builder) is the recursive worker. It examines node->node with std::visit. The visitor uses std::decay_t and if constexpr with std::is_same_v to select the code for each std::variant alternative. This is the C++ counterpart of the TypeScript pass's switch on node.kind. Each branch returns a Fragment; the top-level entry point calls it once and then asks the builder to package the NFA.

```
Automaton thompsonConstruction(const ASTNodePtr& ast) {
NFABuilder builder;
```

```
Fragment fragment = buildFragment(ast, builder);
return builder.build(fragment.start, fragment.accept);
}
```

9.4 Rule one: a character

For a CharNode holding c, the code allocates two states and adds one labeled transition from the first to the second. The returned fragment uses those states as its start and accept.

```
start --c--> accept
```

A path through this fragment consumes exactly the one character c. For the input pattern a, the deterministic allocation order gives state 0 as start and state 1 as accept, with transition 0 --a--> 1.

9.5 Rule two: concatenation

For ConcatNode, buildFragment first builds the left child and then the right child. It adds one epsilon edge from left.accept to right.start. The combined fragment starts where the left child starts and accepts where the right child accepts. Concatenation allocates no wrapper states.

```
left.accept --eps--> right.start
```

Why this recognizes concatenation: to reach the combined accept state, a run must first complete the left fragment, take the epsilon edge without consuming input, and then complete the right fragment. Thus the consumed strings are exactly a string from the left language followed by a string from the right language: L(left)L(right).

For ab, the a fragment is 0 --a--> 1 and the b fragment is 2 --b--> 3. ThompsonPass adds 1 --eps--> 2. The overall start is 0 and the accept is 3, so the accepted string is ab.

9.6 Rule three: alternation

For AltNode, the code recursively builds the left branch, then the right branch. It allocates a fresh start and accept after both branches are complete. Four epsilon edges form a fork at the new start and a join at the new accept: two edges enter the alternatives, and two edges leave their accepts.

```
eps → left fragment → eps
new start new accept
eps → right fragment → eps
```

The run can choose either branch without consuming input, then reach the common accept after that branch succeeds. Therefore the recognized language is the union L(left) union L(right). The NFA expresses the choice by having two epsilon destinations; it does not choose one branch during construction.

9.7 Rule four: Kleene star

For StarNode, the code first builds the child fragment and then allocates a fresh start and accept for the star fragment. Four epsilon edges provide two choices at the start and two choices after each successful child run.

• The new start can enter the child fragment, beginning a repetition.

• The new start can go directly to the new accept, allowing zero repetitions and therefore the empty string.

• After the child accepts, an epsilon edge can return to the child start, allowing another repetition.

• After the child accepts, another epsilon edge can go to the new accept, ending the repetitions.

These routes implement L(child)*: zero or more consecutive strings from the child language. Because the loop is an epsilon edge, it consumes no character itself; each positive repetition consumes whatever the child fragment consumes.

9.8 Exact trace: a|b and (a|b)*

The tests assert exact state numbers as well as transition structure. For a|b, the left a fragment is built first (states 0 and 1), then b (states 2 and 3). Alternation then allocates wrapper start 4 and accept 5. The result is:

```
4 --eps--> 0 0 --a--> 1 1 --eps--> 5
4 --eps--> 2 2 --b--> 3 3 --eps--> 5
```

State 4 is the only start state; state 5 is the only accept state. A run can consume a and reach 5, or consume b and reach 5. No epsilon edge consumes input.

For (a|b)*, the inner alternation uses states 0–5 exactly as above. The outer star allocates start 6 and accept 7, then adds 6 --eps--> 4, 6 --eps--> 7, 5 --eps--> 4, and 5 --eps--> 7. The first pair enters the repeated expression or skips it; the second pair loops for another repetition or exits. This accepts the empty string, a, b, aa, ab, ba, and so on.

9.9 Correctness and complexity

The construction is correct by structural induction on the AST. At a character leaf, the fragment accepts exactly that one character. Assuming the child fragments recognize their AST languages, concatenation forces a successful left run followed by a successful right run; alternation permits either successful branch; and star permits zero or more successful child runs. Each rule therefore preserves the language denoted by its AST node. Applying the argument from leaves to root shows that the complete NFA recognizes the original regular expression's language.

Let C be the number of character nodes, A the number of alternation nodes, S the number of star nodes, and K the number of concatenation nodes. The implementation creates exactly 2C + 2A + 2S states: each character gets two, each alternation gets two wrapper states, each star gets two wrapper states, and concatenation gets none. It adds C + 4A + 4S + K transitions. Both state and transition counts are linear in AST size, so construction time and output space are O(n) for n AST nodes.

The recursive worker uses call-stack space proportional to AST height. The parser's input and nesting limits help guard against pathological patterns, but the construction itself does not rebalance the AST. State IDs follow a deliberate depth-first order (left subtree before right subtree; wrapper states after children). The unordered transition maps do not promise iteration order, so the tests sort/flatten maps when comparing them.

9.10 Pass boundaries, tests, and files

thompsonConstruction is pure with respect to its input: it does not modify the AST, and its NFABuilder is local to the call. Parsing errors belong to ParserPass and are not handled here. This function assumes it receives a valid AST from the parser; it is not a second syntax checker.

The C++ tests call the real parser first, then this pass, and check state counts, start and accept states, and exact transition maps for a, ab, a|b, a*, and the composition (a|b)*. They are structural golden tests: they confirm the generated automaton shape and the deterministic allocation order. The TypeScript tests mirror these cases.

• engine/passes/thompson-pass.cpp — NFABuilder, Fragment, recursive std::visit dispatch, and all four rules.

• engine/include/regexlab/passes/thompson-pass.hpp — public thompsonConstruction signature.

• engine/include/regexlab/types.hpp — AST variants and shared Automaton representation.

• engine/passes/thompson-pass.ts — TypeScript reference implementation.

• tests/thompson_pass_test.cpp and engine/__tests__/unit/thompson-pass.test.ts — exact construction examples and composition checks.

The next pipeline step is SubsetConstructionPass. It starts with the epsilon-closure of the Thompson NFA's start state, then computes DFA transitions for input symbols. Keeping that operation in the next pass makes the compiler pipeline explicit: AST-to-ε-NFA first, ε-NFA-to-DFA after.

10. SubsetConstructionPass — Rabin–Scott subset construction

Compiler design and Theory of Computation perspective

Source focus: engine/passes/subset-construction-pass.cpp and engine/include/regexlab/passes/subset-construction-pass.hpp. The matching TypeScript reference is engine/passes/subset-construction-pass.ts. Tests are in tests/subset_construction_pass_test.cpp and engine/__tests__/unit/subset-construction-pass.test.ts.

10.1 Where this pass fits

SubsetConstructionPass is the third engine stage. ParserPass produces an AST; ThompsonPass turns that AST into an epsilon-NFA; subsetConstruction(const Automaton& nfa) turns the NFA into a DFA. It returns the same general Automaton structure, but its transitions have no eps labels and at most one destination for any state and input symbol.

This pass is the Rabin–Scott construction, also called the powerset or subset construction. Its key representation choice is that one DFA state represents a set of NFA states. The set records every NFA state the NFA could occupy after reading the same input prefix, including states reachable using epsilon transitions.

The NFA is nondeterministic because it can have several outgoing transitions with the same label and can move without consuming input. The DFA simulates all those possible NFA runs together inside one set-valued state. It does not guess one branch and risk losing a valid path.

10.2 The mathematical transition rule

For an NFA state set S and an input symbol x, move(S, x) is the set of destinations reachable from any state in S by consuming exactly x. epsilonClosure(S) is S together with every state reachable from S by zero or more eps transitions.

```
DFA start state = epsilonClosure({NFA start state})
DFA transition on x: S --x--> epsilonClosure(move(S, x))
```

The closure is needed both before the first character and after every consumed character. In this implementation, each stored DFA state is already epsilon-closed. Therefore the loop can call move on that set directly, then close the destinations to form the next set.

This is why epsilon transitions disappear from the DFA. They are followed as part of computing a set before or after a character move; the resulting DFA edge itself is labeled only with an actual input symbol.

10.3 epsilonClosure: follow every free move safely

epsilonClosure starts by copying the input states into a result set and putting those states on a stack. It repeatedly pops one state, looks up its eps destinations in nfa.transitions, and adds any destination not already in the result. A newly discovered state is pushed so its own epsilon edges are explored too.

The membership check is essential. Thompson NFAs can contain epsilon cycles, especially for Kleene star. Since a state is pushed only when it is first inserted, each reachable state is processed at most once during a closure calculation; a loop cannot make the traversal run forever.

States with no outgoing transitions, or with no eps entry, simply contribute no more states. The returned unordered_set has no meaningful iteration order; the algorithm depends on membership, not order.

10.4 move and the alphabet

move scans every NFA state in the supplied set, looks up transitions for the requested symbol, and unions all destination vectors into a result set. It does not follow eps edges. Epsilon handling is kept in epsilonClosure so the two operations have one clear responsibility each.

alphabetOf scans all NFA transition labels, collects the distinct labels, discards the special eps label, and sorts the remaining symbols. Sorting gives the pass a stable symbol order when it discovers subsets and assigns state IDs. For this project's character-based regexes, these labels are the literal input characters present in the pattern.

10.5 State identity and deduplication

stateSets is a vector indexed by DFA state ID. Entry i stores the NFA-state set represented by DFA state i. idBySetKey maps a canonical string representation of a set back to its DFA ID. The local idFor function first builds that key; if it has been seen, it returns the existing ID, otherwise it assigns the next ID and stores the new set.

setKey sorts the NFA state numbers and joins them with commas. Sorting matters because {1, 4, 7} and the same set visited in a different order must produce the same key. Without canonicalization, one mathematical subset could accidentally be entered into the DFA more than once.

The actual state sets are unordered_set values, while the key is order-independent. As a result, epsilon graph traversal order does not change subset identity. DFA IDs are assigned when a subset is first discovered; the initial subset is ID 0.

10.6 Worklist and fixed point

The algorithm first finds the NFA alphabet, then registers epsilonClosure({nfa.startState}) with idFor. That is the DFA start state. A worklist begins with its ID. Each worklist item is a DFA state whose outgoing transitions still need to be computed.

The main loop pops an ID from the back of a vector, so it uses a last-in, first-out worklist. If that ID was already processed, it skips it; otherwise it marks the ID processed and checks every alphabet symbol.

• Compute moved = move(stateSets[current], symbol).

• If moved is empty, omit the transition. The source comment describes this as an omitted implicit dead state.

• Otherwise compute epsilonClosure(moved), then call idFor to reuse or create the target DFA state.

• Record one destination for current and symbol, then put the target on the worklist if it has not yet been processed.

New subsets may be discovered while the loop is running. The vector of subsets grows, and newly discovered IDs are added to the worklist. The loop ends only when every reachable DFA subset has had all of its symbol transitions considered. That is the fixed point: processing known subsets no longer discovers an unprocessed subset.

An ID might be pushed more than once before its first pop if different transitions discover it. The processed set prevents the algorithm from expanding that DFA state more than once. idFor separately prevents assigning two IDs to the same subset.

10.7 Worked example: a|b

Use the Thompson NFA for a|b from the previous chapter. Its start is NFA state 4 and its accept is state 5. Epsilon edges are 4 to 0 and 2, 1 to 5, and 3 to 5; the labeled edges are 0 --a--> 1 and 2 --b--> 3.

First take the epsilon closure of {4}. It contains 4 itself and follows 4's two epsilon edges, so DFA state 0 represents {0, 2, 4}. This one DFA state captures both alternatives before any input has been consumed.

The sorted alphabet is [a, b]. On a, move({0, 2, 4}, a) = {1}; its epsilon closure is {1, 5}, registered as DFA state 1. On b, move({0, 2, 4}, b) = {3}; its closure is {3, 5}, registered as DFA state 2.

```
DFA state 0 = {0, 2, 4} --a--> state 1 = {1, 5}
```

```
DFA state 0 = {0, 2, 4} --b--> state 2 = {3, 5}
```

State 0 is not accepting because its subset does not contain NFA accept state 5. States 1 and 2 are accepting because each contains 5. There are no further labeled transitions, so the generated DFA has three states and exactly the transitions for a and b from state 0. It accepts a or b, matching the NFA language.

10.8 Worked example: a* and the self-loop

For a*, the Thompson NFA has character edge 0 --a--> 1, star start 2, star accept 3, and epsilon edges 2 to 0 and 3, plus 1 to 0 and 3. The initial closure is {0, 2, 3}; it becomes DFA state 0. It is accepting because it contains NFA accept state 3, which correctly allows the empty string.

On a, the NFA move from {0, 2, 3} is {1}. Its closure is {0, 1, 3}, which becomes DFA state 1. This is also accepting. From state 1, another a again moves to {1}, whose closure is the same {0, 1, 3}; idFor reuses state 1, so the DFA gets an a self-loop.

```
DFA state 0 = {0, 2, 3} --a--> state 1 = {0, 1, 3}
DFA state 1 = {0, 1, 3} --a--> state 1 = {0, 1, 3}
```

Both states accept, and the loop allows any number of a characters. The construction has collapsed the NFA's epsilon loop and repetition paths into a compact deterministic cycle.

10.9 Why the construction preserves the language

The central invariant is: after reading a prefix w, the DFA is in the state whose subset is exactly the set of NFA states reachable after the NFA reads w and then follows zero or more epsilon transitions. Initially, this holds because the DFA starts in epsilonClosure({nfa.startState}).

For the induction step, suppose the invariant holds for prefix w and the next character is x. Every possible NFA run that can read x must first be at one of the states in the current closed subset. move collects all destinations reached by consuming x; epsilonClosure then adds every state reachable for free afterward. That is exactly the set represented by the next DFA state. Thus the invariant holds for wx.

A DFA subset is accepting exactly when it contains at least one NFA accept state. This mirrors NFA acceptance: the NFA accepts when at least one possible run reaches an accept state after consuming the whole input. Therefore the DFA and NFA accept the same language.

10.10 Acceptance, determinism, and partial transitions

After all reachable subsets are discovered, the pass scans stateSets. If a subset contains any state from nfa.acceptStates, its DFA ID is inserted into acceptStates. This is a set-intersection test: S is accepting iff S intersection F_NFA is nonempty. A DFA can have multiple accepting states, which is why the shared Automaton type stores a set.

For each DFA state and input symbol, move followed by closure yields one subset, and idFor yields one target ID. The implementation stores that target in a one-element vector to fit the shared Automaton transition shape. No eps transition is emitted. The tests check both of these properties.

This implementation omits transitions when move returns the empty set. In formal presentations, subset construction often includes the empty subset as a sink state so the DFA has a transition for every alphabet symbol from every state. Here, the absent transition represents that dead state implicitly. This is a partial DFA representation; the distinction matters when reading state counts or transition tables.

10.11 Termination and state explosion

An NFA with m states has at most 2^m distinct subsets, including the empty subset. The pass creates only reachable, nonempty subsets because it starts from the initial closure and skips empty moves. Therefore the worklist must terminate after finitely many states, though the number of reachable subsets can still be exponential in m.

Each DFA state is processed once, and for each one the code considers every alphabet symbol. Each operation scans NFA states and their relevant transitions, so the rough work is proportional to the number of reachable DFA subsets times the alphabet size and the NFA traversal cost. The number of stored DFA subsets can be up to 2^m; this is the classic state-explosion risk of converting an NFA to a DFA.

Thompson's construction uses a number of NFA states linear in regex AST size, so subset construction is linear in neither the regex length nor the NFA size in the worst case. This is why the pipeline treats NFA construction, determinization, and later minimization as separate passes: each transformation has a distinct purpose and cost profile.

10.12 Tests and source files

The C++ tests build a real NFA by parsing each pattern and calling ThompsonPass, then call subsetConstruction. They verify the already-deterministic single-character case, alternation folding, the Kleene-star self-loop, one destination per symbol, and the complete removal of eps labels. The TypeScript tests mirror the C++ golden cases.

• engine/passes/subset-construction-pass.cpp — epsilonClosure, move, alphabetOf, setKey, idFor, worklist, and accepting-state detection.

• engine/include/regexlab/passes/subset-construction-pass.hpp — public NFA-to-DFA function signature.

• engine/include/regexlab/types.hpp — shared Automaton representation used by both the NFA and DFA.

• engine/passes/subset-construction-pass.ts — TypeScript reference with the same algorithm and ordering choices.

• tests/subset_construction_pass_test.cpp and engine/__tests__/unit/subset-construction-pass.test.ts — exact DFA structure and determinism checks.

After this pass, MinimizationPass can merge equivalent DFA states. That next transformation is separate: subset construction removes nondeterminism while preserving the language; minimization reduces the number of deterministic states while preserving the language.

11. MinimizationPass — Moore's partition refinement

Compiler design and Theory of Computation perspective

Source focus: engine/passes/minimization-pass.cpp and engine/include/regexlab/passes/minimization-pass.hpp. The matching TypeScript reference is engine/passes/minimization-pass.ts. Tests are in tests/minimization_pass_test.cpp and engine/__tests__/unit/minimization-pass.test.ts.

11.1 Where this pass fits

MinimizationPass is the fourth engine stage: ParserPass creates the AST, ThompsonPass creates an epsilon-NFA, SubsetConstructionPass creates a DFA, and minimize(const Automaton& dfa) merges DFA states that have indistinguishable future behavior. It returns another Automaton and preserves the language recognized by the input DFA.

This is Moore's algorithm, an iterative partition-refinement algorithm. It starts with a broad grouping of states and repeatedly splits a group when its members behave differently. It never merges an accepting state with a non-accepting state, because those states already disagree on whether the empty suffix is accepted.

The input contract matters: this algorithm expects a DFA. It reads the first destination for each state-symbol pair, so it does not determinize an NFA. The preceding SubsetConstructionPass supplies deterministic transitions and omits epsilon labels.

11.2 What it means for two DFA states to be equivalent

Two DFA states are equivalent when no possible remaining input string can distinguish them: starting from either state, every suffix must either be accepted from both or rejected from both. If some suffix leads one state to acceptance and the other to rejection, that suffix distinguishes the states, so they cannot be merged.

A useful one-step characterization is recursive. Equivalent states must agree on acceptance, and for every alphabet symbol their transitions must lead to equivalent states. Moore refinement makes this condition concrete by recording which current partition block each transition reaches.

This connects to the Myhill–Nerode view of regular languages: a minimal DFA has one state per equivalence class of prefixes that have the same possible accepting continuations. The pass computes those behavior classes from the supplied DFA structure.

11.3 Starting partition: accepting versus non-accepting

The code creates all state IDs from 0 through stateCount - 1, then divides them into accepting and non-accepting vectors. Empty vectors are skipped. This is the initial partition P0, the coarsest safe grouping: states in different groups already have different acceptance behavior.

For example, if a DFA has states {0, 1, 2, 3} and only state 3 accepts, the starting blocks are {0, 1, 2} and {3}. States 0, 1, and 2 may or may not be equivalent; refinement discovers that. State 3 cannot join them.

11.4 refine(): split blocks using transition signatures

Each refinement round first builds groupIndexOf, mapping every current state to the index of the block that contains it. Then it examines every state in every block and builds a signature, one entry per alphabet symbol in sorted order.

• If the state has a transition on a symbol, that signature entry is the current block index of the destination.

• If the transition is missing, the signature entry is the special marker '-'. This pass does not create an explicit sink state.

States in one old block with identical signatures stay together. States with different signatures are placed in separate buckets. The code records the order in which each signature first appears in bucketOrder; the unordered_map stores the buckets, while bucketOrder preserves their creation order when newGroups is assembled.

The block is split when it produces more than one signature bucket. refine returns both the newly ordered blocks and a changed flag. A signature uses destination block IDs, not raw destination state IDs: two destinations are considered alike at this round when the current partition still considers them alike.

11.5 Repeat until a fixed point

minimize calls refine repeatedly while changed is true. A split in one round can make a later distinction visible: once two destination states are separated, states that transition to those destinations may need to split in the next round. Repeating carries distinctions backward through paths of increasing length.

The algorithm stops after a complete round that makes no split. At that fixed point, all states in each block have the same acceptance status and, for every symbol, transition into the same block or share the same missing transition. No further behavioral distinction can split a block.

11.6 Worked example: minimizing (a|b)*ab

The C++ test uses (a|b)*ab as a classic example. Subset construction produces four DFA states with these transitions; state 3 is the only accepting state. Missing entries mean there is no transition.

```
state 0: a → 1, b → 2
state 1: a → 1, b → 3
state 2: a → 1, b → 2
state 3: a → 1, b → 2 (accepting)
```

The alphabet is [a, b]. The initial partition is P0 = {{0, 1, 2}, {3}}. In the first block, use destination block indices 0 for non-accepting and 1 for accepting. The signatures are:

```
state 0: (0, 0)
state 1: (0, 1)
```

```
state 2: (0, 0)
state 3: (0, 0) — it remains in its separate accepting block
```

States 0 and 2 have the same signature, while state 1 differs because b reaches the accepting block. The partition becomes P1 = {{0, 2}, {1}, {3}}. State 3 remains separate even though its transition signature matches 0 and 2, because the starting partition already separated acceptance status.

On the next round, the blocks are indexed 0 for {0, 2}, 1 for {1}, and 2 for {3}. States 0 and 2 both have signature (1, 0): a leads to {1}, and b leads back to {0, 2}. No block splits, so the algorithm has reached a fixed point.

Each final block becomes one minimized state. Starting from the block containing old state 0 gives this result, which matches the C++ golden test:

```
new 0 = {old 0, old 2}: a → 1, b → 0
new 1 = {old 1}: a → 1, b → 2
new 2 = {old 3}: a → 1, b → 0 (accepting)
```

The DFA shrinks from four states to three. States 0 and 2 merge because every future suffix behaves the same from both. The language is unchanged: the minimized DFA still recognizes strings over {a,b} that end in ab.

11.7 Building the quotient DFA

After refinement, groupIndexOfState maps every original state to its final block. A block is one equivalence class and becomes one state in the quotient DFA. The start state is the block containing dfa.startState.

For each block, the code chooses groups[groupIndex][0] as a representative. This is valid because the fixed-point condition guarantees all members transition to the same blocks for each symbol. It looks up each representative transition, maps its destination to a final block, and emits one transition to that block.

Canonical output IDs are assigned by a worklist starting at the start block. idFor gives a block its ID the first time it is discovered; the alphabet is sorted, and the worklist is a vector processed from the back. This makes numbering stable for the project's golden tests. The explicit bucketOrder also preserves TypeScript Map insertion order when ported to C++, where unordered_map has no stable iteration order.

An output block is accepting if its representative is in dfa.acceptStates. Since the first partition separated accepting from non-accepting states, every final block is uniform on acceptance. The new Automaton contains the canonical state count, new start ID, accepting IDs, and quotient transitions.

11.8 Why the minimized automaton preserves the language

The initial partition groups only states with the same immediate acceptance result. Every refinement preserves that property and splits states when some input symbol leads to different current blocks. If destinations are separated because a suffix distinguishes them, then the symbol followed by that suffix distinguishes their source states; refinement propagates this fact backward.

At the fixed point, equivalent states have the same acceptance status and corresponding transitions lead to equivalent blocks. Replacing every block by one representative state therefore preserves every possible run's sequence of behavior classes and preserves acceptance for every input string.

The resulting blocks cannot be merged further without combining states that are distinguishable by some suffix. For a DFA under the algorithm's input conventions, these stable blocks are the behavioral equivalence classes the minimal machine needs.

11.9 Reachability, missing transitions, and input assumptions

The TypeScript implementation documents that its input states are reachable, which is true for the preceding subset-construction pass: that pass creates states only by following transitions from its start state. The minimizer's final worklist also assigns output IDs only to blocks reached from the start block, so unreachable quotient blocks do not appear in its output.

The DFA from subset construction can be partial because empty moves are omitted instead of creating an explicit empty-subset sink. In refine, a missing transition is represented by '-', which is distinct from a transition to any real block; the pass does not materialize or merge a virtual sink. That matches the project's partial-DFA pipeline. If this function is reused with arbitrary incomplete DFAs containing explicit dead states, that representation detail can affect whether equivalent states merge; a conventional complete-DFA minimizer first adds a sink for all missing transitions.

The function assumes determinism and indexes the first destination in each transition vector. It also assumes state IDs are the contiguous range 0 to stateCount - 1, as produced by the preceding pass. These are the engine's shared Automaton conventions, not checks performed by minimize.

11.10 Complexity and the Moore versus Hopcroft choice

Let n be the number of DFA states and k the alphabet size. One refine round computes a signature for each state by checking each symbol, so its work is O(nk), aside from map and string overhead. Each round that changes the partition creates at least one additional block, so there can be at most n - 1 splitting rounds, followed by one stable round. This gives the usual $O(n^2k)$ time bound for this implementation.

The partition and signature buckets use O(n) state membership plus the transition/result storage; the output DFA has at most n states. In contrast, Hopcroft's partition-refinement algorithm can achieve $O(n \log n \cdot k)$ with a more involved splitter worklist. The project intentionally implements Moore's simpler version so each refinement round and its correctness are easier to trace.

11.11 Tests and source files

The C++ tests build a pattern through parsing, Thompson construction, subset construction, and minimization. They verify the four-to-three reduction for (a|b)*ab, merging of equivalent accept states in ab|ac, preservation of an already minimal aa|a DFA, and the invariant that accepting and non-accepting states do not merge. The TypeScript tests mirror these cases.

• engine/passes/minimization-pass.cpp — alphabet extraction, signature refinement, fixed-point loop, canonical renumbering, and output construction.

• engine/include/regexlab/passes/minimization-pass.hpp — public minimize function signature.

• engine/include/regexlab/types.hpp — shared Automaton type.

• engine/passes/minimization-pass.ts — TypeScript reference and Moore algorithm notes.

• tests/minimization_pass_test.cpp and engine/__tests__/unit/minimization-pass.test.ts — language-preserving merges and golden minimized DFAs.

After minimization, MatcherPass runs the DFA over a candidate string and can produce the trace shown by the application. The stages remain separate: subset construction removes nondeterminism, Moore refinement merges equivalent deterministic states, and matching executes the resulting DFA.

11.12 Why Moore's algorithm was chosen: advantages and limitations

Why it fits this project: Moore's algorithm makes minimization visible as repeated partition refinement. Starting with accepting and non-accepting blocks, each round separates states only when their transition behavior proves them distinguishable. That is easier to trace by hand and to defend in a course explanation than a more optimized splitter-worklist algorithm.

The project notes explicitly put Moore's algorithm in Core and Hopcroft's algorithm in Hard Stretch: Hopcroft is considered only after the simpler method is solid and explainable. That choice prioritizes a correct, testable pass with clear intermediate partitions over asymptotically faster minimization.

• Advantages: the initial partition has an immediate correctness reason; signatures show exactly why states split; the fixed-point condition is easy to test; stable blocks directly define the quotient DFA; the implementation is compact and mirrors the TypeScript reference.

• Limitations: Moore refinement takes O(n² · k) time for n DFA states and alphabet size k, compared with Hopcroft's O(n log n · k). It can become slow if determinization has already produced a large DFA. The implementation expects a deterministic automaton with contiguous state IDs and preserves the pipeline's partial-transition convention rather than inserting a sink.

So Moore is a strong fit for the current educational scope and input limits, while Hopcroft is the natural optimization if profiling shows minimization is a bottleneck or the supported pattern range grows.

12. MatcherPass — DFA simulation

MatcherPass is the fifth and final pass. It receives the minimized DFA and the test string, then returns a MatchTrace. The earlier passes have already resolved the parser structure, epsilon choices, nondeterminism, and equivalent states. Matching is therefore a deterministic walk: at each character there is at most one next state.

12.1 The exact input and output

C++ entry point: match(const Automaton& dfa, const std::string& input), declared in engine/include/regexlab/passes/matcher-pass.hpp and implemented in engine/passes/matcher-pass.cpp. runPipeline passes minDfa to this function, so the trace follows the minimized machine, not the original NFA or pre-minimization DFA.

The result contains steps, result (Match or NoMatch), and an optional failurePosition. Each MatchStep stores the consumed character and activeStates. Because the input automaton is deterministic, each recorded activeStates vector contains exactly one state: the state reached after consuming that character.

12.2 Reading matcher-pass.cpp line by line

1. Initialize current to dfa.startState. Before input is consumed, the machine is at its start state.

2. Iterate i from zero to input.size() - 1. Read input[i], make a one-character transition key, and look up the current state's transition for that symbol.

3. The Automaton contract stores transition destinations in a vector because NFAs can have several destinations. A DFA has at most one destination for a given state and symbol, so MatcherPass takes the first destination. It relies on subset construction and minimization to have supplied a deterministic machine.

4. If the state or symbol has no usable destination, the machine has reached a dead end. Return NoMatch with failurePosition = i. The failing character is not added to steps; the trace contains only characters consumed before the failure.

5. Otherwise assign the destination to current and append MatchStep{character, [current]}. The snapshot is post-transition: it tells the visualizer which state is active after that character.

6. If the loop consumes the whole string, accept exactly when current belongs to dfa.acceptStates. A missing failurePosition here means no transition failed; it does not by itself mean the string matched.

12.3 Full-string language recognition

This is full-string recognition. The machine starts at the beginning of the supplied input and accepts only if the state reached after consuming every character is accepting. It does not search for a matching substring or return the first matching span. The parser has no ^ or $ tokens because full-input behavior is supplied by the matcher itself. The differential tests compare against std::regex_match in C++ and anchor the JavaScript oracle to force the same whole-string semantics.

12.4 Worked traces

Pattern ab, input ab: start at state 0; read a and record state 1; read b and record state 2; if state 2 is accepting, return Match. The C++ and TypeScript tests pin this exact two-step trace for the current minimized machine.

Pattern ab, input ac: read a and record state 1. At input index 1, no c transition exists, so return NoMatch with failurePosition 1. The trace has one step; c was never consumed.

Pattern ab, input a: read a and reach state 1, then input ends. If state 1 is not accepting, return NoMatch with no failurePosition. This differs from a dead end: the machine had a valid transition for every character it received.

Pattern a*, input empty: the loop executes zero times. The start state is accepting, so the empty string is accepted and steps is an empty array. For input aaaa, each character loops back to the one accepting state, producing four steps.

Pattern (a|b)*ab recognizes strings over {a,b} that end in ab. For example ab, aab, and bbab are accepted; ba is rejected after consuming both characters because the final state is not accepting. That rejection is not a dead end.

12.5 Complexity and trace cost

For an already-built DFA, the recognition walk visits at most one transition per input byte, so time is O(n) in input length under the usual average-constant-time hash-map lookup assumption. If only the final yes/no answer were needed, the matcher would keep only current and use O(1) extra state.

This implementation deliberately records every successful transition. The trace therefore takes O(n) output space, with one MatchStep per consumed character. The implementation is linear in the input length, but it is not O(1) total space when tracing is enabled. The DFA can also be much larger than the NFA because subset construction can create exponentially many states; that cost happens before matching.

12.6 Why this matching method was chosen

The DFA walk is the natural endpoint of this project's preceding passes. Thompson construction gives a compositional NFA; subset construction removes nondeterminism; minimization merges equivalent states. Once that machine exists, recognition needs no recursive search or branch stack.

Advantages: one visible transition per input character; predictable linear matching time; no regex-engine backtracking path to hide; simple trace semantics; and a direct connection between DFA acceptance and regular-language membership. The same loop is small enough to test with exact traces.

Limitations: it recognizes only languages in the supported regular-expression subset; it cannot implement backreferences or other features that require richer engine semantics. Building the DFA can cause state explosion, and the trace consumes linear memory. It checks the entire string rather than searching within a larger string. For a production regex workbench, this is a different goal from reproducing PCRE, JavaScript, or Python engine behavior.

12.7 Tests and files

The matcher tests cover exact acceptance, dead-end rejection, rejection after consuming all input, empty input, a self-loop, and a language whose rejected string never dead-ends. There is a TypeScript counterpart and a C++ test. Root tests/matcher_pass_test.cpp is the most direct review target; tests/differential_test.cpp checks final answers against std::regex_match over generated patterns and strings.

Key files: engine/passes/matcher-pass.cpp; engine/passes/matcher-pass.ts; engine/include/regexlab/passes/matcher-pass.hpp; engine/__tests__/unit/matcher-pass.test.ts; tests/matcher_pass_test.cpp; engine/include/regexlab/types.hpp (MatchStep and MatchTrace); engine/pipeline.cpp (passes minDfa to match).

13. Pipeline contracts and the JavaScript/WASM boundary

The five passes do not communicate through arbitrary UI objects. They use explicit engine data types. runPipeline is the orchestration layer; it is plumbing around the passes rather than a sixth algorithm.

13.1 The shared automaton representation

Automaton in engine/include/regexlab/types.hpp contains stateCount, startState, acceptStates, and transitions. Transitions are represented as state -> symbol -> vector of destination state IDs. The vector is needed for an NFA; for a DFA the pipeline expects at most one destination per state and symbol. The special symbol eps denotes an epsilon edge in the NFA.

The subset pass omits empty moves rather than creating an explicit empty-subset sink. The DFA is therefore partial. A missing edge means that this path rejects as soon as that symbol is read. The minimizer and matcher follow this convention. State IDs are integers and the passes assume the contiguous IDs produced by their predecessor.

13.2 What runPipeline preserves

runPipeline(pattern, input) calls parse, thompsonConstruction, subsetConstruction, minimize, and match in that order. If parsing fails, it returns immediately with ok = false and the ParseError; the four later passes do not run. On success it returns ok = true plus the AST, NFA, DFA before minimization, minDfa, and MatchTrace.

These are snapshots from the actual pass outputs. The caller can display the intermediate representations without re-running each stage or trying to reconstruct missing algorithm state from the final match. The core invariants to defend are: each construction preserves the language of the input representation; minimization merges only behaviorally equivalent DFA states; and the matcher accepts precisely when the full input ends in an accept state.

13.3 One function across the WASM boundary

engine/json_bridge.cpp implements runPipelineJson(pattern, input). It calls runPipeline, converts either the error or all successful snapshots to JSON, and returns one JSON string. engine/wasm/bindings.cpp is thin Embind glue exposing that function as Module.runPipeline(pattern, input). JavaScript parses the returned string. CMake includes the Emscripten binding target only for an Emscripten build.

The success JSON has ok, ast, nfa, dfa, minDfa, and trace fields. The error JSON has ok = false and an error object; it does not contain a trace. Integer state IDs stay integers in arrays, but transition-map state IDs become string keys because JSON object keys are strings. Null failurePosition means the matcher consumed the whole input without a missing transition.

This boundary keeps the algorithm implementation in the native-buildable C++ engine and avoids exposing C++ containers or one binding per pass to the browser. json_bridge.cpp can be built and tested natively without Emscripten; bindings.cpp is the only file that requires the Emscripten API.

13.4 Cross-language and boundary confidence

The TypeScript engine is retained as a reference implementation. The project has unit tests for both tracks, shared parser-case fixtures, a C++ JSON-bridge test that parses and checks the actual JSON structure, and differential tests. The differential suites generate 200 valid patterns and 10 inputs per pattern (2,000 pairs) and compare whole-string decisions with native regex oracles.

This is strong practical evidence for the sampled grammar and input sizes, not a mathematical proof of equivalence for every possible pattern and string. The random tests exercise generated trees only up to a small depth and short inputs; individual edge-case tests are still needed for parser errors, limits, trace details, and serialization.

13.5 Files to inspect for an end-to-end review

engine/include/regexlab/types.hpp defines the AST, Automaton, ParseResult, and MatchTrace contracts. engine/include/regexlab/pipeline.hpp defines PipelineResult and the public function. engine/pipeline.cpp wires the five passes. engine/json_bridge.cpp serializes the result. engine/include/regexlab/json_bridge.hpp declares the boundary function. engine/wasm/bindings.cpp exposes it to JavaScript. CMakeLists.txt separates the native engine/tests from the optional WASM target.

14. Scope and limitations to state clearly

RegexLab is an educational regular-language compiler and recognizer for a deliberately small syntax, not a drop-in implementation of a production regex flavor. The current parser handles literal characters, implicit concatenation, | alternation, * Kleene closure, parentheses, and backslash escapes for metacharacters.

Do not mistake EBNF notation such as '*'? for support for the regex question-mark quantifier: in that grammar, ? means the grammar item may be absent. The implemented regex operators are not the same as the grammar's meta-notation.

The parser caps patterns at 500 ASCII units. It also has a 1,000-level nesting limit, but under the default 500-unit length limit the depth limit is not reachable; the depth guard is defense in depth and can be exercised with custom ParseLimits in tests. Over-limit patterns return PatternTooComplex rather than crashing.

The current syntax does not provide production features such as character classes, dot wildcard, + or ? quantifiers, anchors, flags, captures, lookarounds, or backreferences. Pattern characters are ASCII-validated. Matching is whole-string and byte-oriented through std::string; it is not a Unicode-aware flavor implementation.

The determinized DFA can grow exponentially in the worst case. Moore minimization is easier to trace than Hopcroft's algorithm but uses O(n^2 k) time for n DFA states and alphabet size k. The 500-unit pattern cap is a practical guard informed by the cost of later stages, not proof that every accepted input has a small DFA.

These constraints are review strengths when presented honestly: the small grammar makes the algorithms visible and testable. They are limitations if the goal is compatibility with everyday JavaScript, Python, Java, or PCRE expressions.

15. Review preparation

15.1 A concise project explanation

RegexLab takes a pattern from a small regular-expression grammar, parses it into an AST, builds an equivalent epsilon-NFA with Thompson's construction, determinizes it with Rabin–Scott subset construction, minimizes the DFA with Moore refinement, and simulates the minimized DFA on the entire input. The C++20 pipeline returns snapshots from every stage and a character-by-character trace. A JSON bridge exposes the same pipeline to JavaScript through one Emscripten function.

15.2 Questions a reviewer may ask

Why not match directly with JavaScript RegExp?

A native regex library would hide the automata conversions that are the learning objective. The project constructs the machine explicitly, so a reviewer can inspect each intermediate representation. The trade-off is a smaller syntax and no claim of production-flavor compatibility.

Why use an NFA before a DFA?

Thompson construction maps each AST operation to a small compositional NFA fragment. It avoids making local parser choices depend on deterministic state sets. Subset construction then turns epsilon-closed sets of NFA states into deterministic states.

Why minimize after subset construction?

Determinization can create states that have identical future behavior. Moore refinement groups states by acceptance and repeatedly splits them when a symbol leads to different groups. The quotient machine recognizes the same language with fewer states when equivalent states exist.

Is this a substring search?

No. MatcherPass starts at the start state, consumes every input character, and then checks the final state. This is full-string recognition, equivalent to an anchored match for the supported subset.

Does O(1) space describe the actual traced implementation?

Only the recognizer's live state is constant-sized. Since the implementation stores one MatchStep for each successful transition, MatchTrace uses O(n) output memory.

How do we know the ports agree?

The repository has matching pass-level examples, shared parser fixtures, C++ JSON-bridge structure tests, and C++/TypeScript differential suites over generated patterns and strings. These provide bounded evidence; they do not replace proofs or exhaustive testing.

What happens with invalid syntax?

ParserPass returns a structured ParseError with a type and source position. runPipeline short-circuits before automaton construction; the JSON response has ok = false and an error object.

What happens if determinization becomes large?

Subset construction has a worst-case exponential state count. The project bounds pattern length and chooses readable Moore refinement for the current educational scope. Larger supported inputs would require profiling, resource budgets, and potentially a faster minimizer or alternate construction.

15.3 A useful demonstration pattern

Use (a|b)*ab to show all stages and then test bbab (accepted) and baba (rejected). It demonstrates alternation, closure, concatenation, state-set determinization, minimization, and a final DFA trace. Explain that state IDs and layouts are implementation details; the language and transition behavior are the semantic facts.

For a precise matcher contrast, also show ab against inputs ab, a, and ac. These separate acceptance, an incomplete prefix that ends in a non-accepting state, and a true dead end with a failure position.

16. Existing tools, what RegexLab contributes, and what not to claim

16.1 Similar work already exists

Yes. Regex visualization and automata conversion are established ideas. JFLAP is a formal-languages teaching package: its official site lists regular-expression-to-FA, NFA-to-DFA, and DFA-to-minimal-DFA conversions, and its course materials describe automata simulation [S5, S6]. It is a serious comparator, not something to dismiss.

There are browser regex tools with different goals. Regex101 supports several real-world engine flavors, explains syntax, runs tests locally, and offers a PCRE2 debugger that traces matching operations and backtracking [S1, S2]. RegExr supports JavaScript and PCRE, live match feedback, tests, and a plain-English explanation [S3]. Regexper presents a regular expression as a railroad-style syntax diagram [S4]. These tools are valuable for authoring and debugging actual regexes.

There are also small web projects that overlap closely with RegexLab's idea. The regex-to-automata-visualizer README describes Regex -> NFA -> DFA -> minimized DFA, step-by-step conversion modules, and live string playback [S7]. The regex-fsm README describes Thompson construction, subset construction, Moore minimization,

synchronized views, and step-by-step simulation [S8]. So RegexLab should not claim to be the first, the only, or the only visualizer of this pipeline.

16.2 Is visualization rare?

Not in the broad sense. Regex debuggers, railroad diagrams, automata teaching packages, and browser-based automata projects all visualize something. A defensible, narrower observation is that the most familiar regex testers usually foreground practical matching against a chosen engine, while the automata pipeline is more often presented by formal-languages teaching tools or specialized projects. The exact end-to-end view exists, but it is less central to mainstream regex tester workflows. This is a qualitative comparison of published feature descriptions, not a market-size measurement.

16.3 What RegexLab can credibly present as its difference

RegexLab's distinction is its implementation contract and reviewability: five named passes are implemented in the C++20 engine; runPipeline carries the AST, NFA, DFA, minimized DFA, and trace as real snapshots; and a single JSON function is exposed to JavaScript through WASM. The TypeScript prototype, C++ tests, shared fixtures, bridge tests, and differential tests give a reviewer concrete places to inspect the same behavior.

That makes RegexLab especially useful for explaining why each compiler/TOC transformation is valid and what the matcher does with one input. It can teach the internal finite-automaton model rather than merely show whether a production engine accepted some text. Its advantage is pedagogical transparency and an inspectable implementation, not a larger regex feature set.

16.4 Compared with standard regex checks

A standard regex tester is better when the task is to write a production pattern, check a particular runtime's exact flavor, use captures or replacements, diagnose lookarounds/backreferences, or investigate a real engine's backtracking. Regex101 and RegExr document broader engine and test workflows than this project [S1-S3].

RegexLab is better suited to a compiler-design or Theory of Computation review where the question is how the source language becomes an AST, how an NFA is determinized, why DFA states can merge, and how a final machine accepts or rejects a complete string. The comparison should be framed as complementary tools serving different jobs, not a claim that RegexLab replaces or universally outperforms mature regex testers.

16.5 Strengths, limitations, and review claims

Strong claims supported by this repository: each stage has a named boundary and source file; successful runs retain intermediate snapshots; matching follows the minimized DFA; the WASM surface is one function returning JSON; and the repository contains pass tests, parity fixtures, bridge tests, and bounded differential checks.

Claims to avoid: that no equivalent visualizer exists; that the engine supports standard regex syntax; that the DFA is always smaller or construction is always fast; that one set of 2,000 sampled differential cases proves correctness; or that the interface is more usable than competitors without a usability study. Public project descriptions establish feature overlap, but they do not support a benchmark of correctness, speed, accessibility, or learning outcomes.

A strong review can show one pattern through all five passes, explain the preserved-language invariant at each conversion, demonstrate both accepted and rejected inputs, and acknowledge the exact syntax and complexity limits. That is a more credible case than claiming broad superiority.

16.6 Sources reviewed on 20 October 2026

[S1] regex101 documentation, Choose a regex engine: https://support.regex101.com/editor/engines/

[S2] regex101 documentation, Step through a regex: https://support.regex101.com/application/regex101-pro/regex-debugger/

[S3] RegExr official site: https://regexr.com/

[S4] Regexper official site: https://regexper.com/

[S5] JFLAP, What Is JFLAP?: https://www.jflap.org/whatis.html

[S6] JFLAP, Instructor Use: https://www.jflap.org/instructorUse.html

[S7] Sainath-10, regex-to-automata-visualizer README: https://github.com/Sainath-10/regex-to-automata-visualizer

[S8] Abdalla-Eldoumani, regex-fsm README: https://github.com/Abdalla-Eldoumani/regex-fsm

Comparison method: the product documentation and project READMEs support the feature descriptions above. This desk review did not run competing applications side by side, audit their source code, benchmark performance, or measure learning outcomes. The external descriptions can change after the review date.